

COSC 612/AIT 624

Agile Software Development

Dr. Akshita Maradapu Vera Venkata Sai
Department of Computer and Information Sciences
Towson University

Agile Methods

What if we expect **change**?



How the customer explained it



How the project leader understood it



How the engineer designed it



How the programmer wrote it



How the sales executive described it



How the project was documented



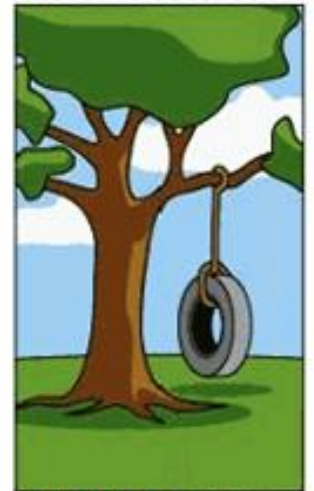
What operations installed



How the customer was billed



How the helpdesk supported it



What the customer really needed

Agile Methods

Requirements **are not**
understood

Don't build this ...

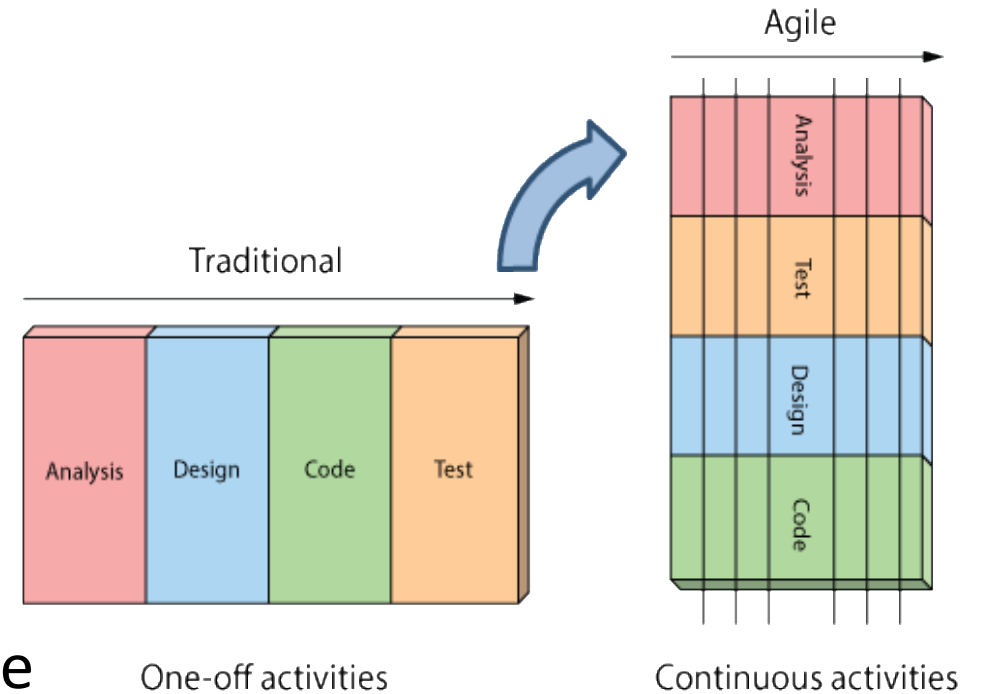
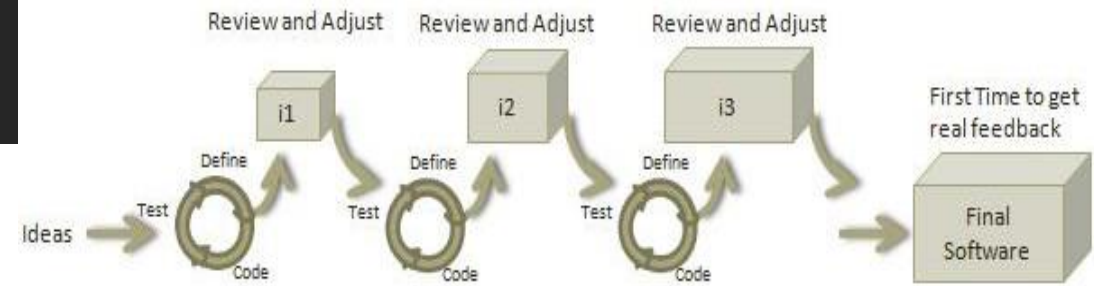


if all you need is this.



Agile Development

- The system is developed as
 - a series of versions or increments
 - with stakeholders involved in version specification and evaluation
- Frequent delivery of new versions for evaluation
- Extensive tool support used to support development.
 - e.g. automated testing tools
- Minimal documentation – focus on working code

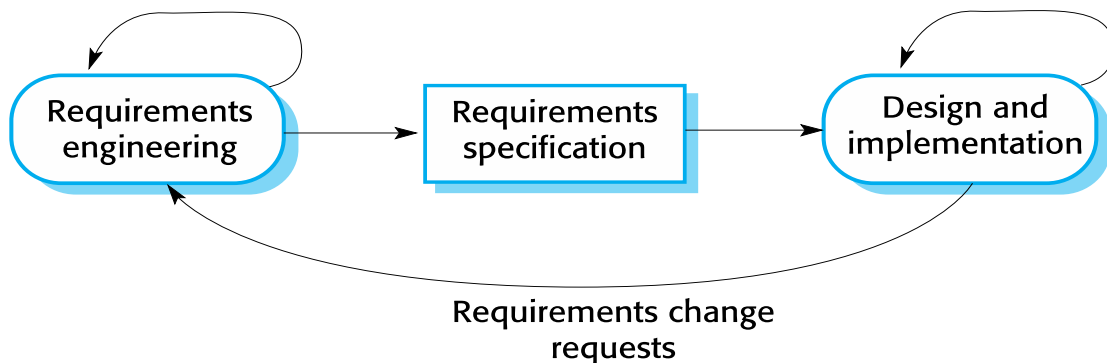


Plan-driven and Agile Development

Plan-driven development

- Software engineering is based around
 - separate development stages
 - outputs to be produced at each of these stages planned in advance.
- Not necessarily waterfall model –
 - incremental development is possible.
- Iteration occurs within activities.

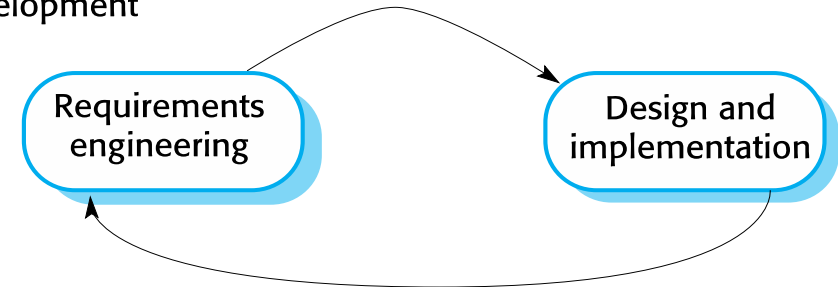
Plan-based development



Agile development

- Specification, design, implementation and testing are inter-leaved
- The outputs from the development process are decided through:
 - a process of negotiation during the software development process.

Agile development



Rapid Software Development

- Rapid software development and delivery is now often the most important requirement for software systems
 - Businesses operate in a **fast –changing requirement** environment
 - it is practically impossible to produce a set of **stable software requirements**
 - **Software has to evolve quickly** to reflect changing business needs.
- Plan-driven development is essential for some types of system
 - but does not meet the above business needs
- Agile development methods emerged in the late 1990s whose aim was to
 - radically reduce the delivery time for working software systems

Agile Methods

- The aim of agile methods is to :
 - reduce overheads in the software process
 - e.g. by limiting documentation
 - respond quickly to changing requirements
 - without excessive rework.
- **How?**

By following certain Agile Principles

The Principles of Agile Methods

- Customer involvement
- Incremental delivery
- People not process
- **Embrace change**
- Maintain simplicity
- Product development
- Custom system development within an organization

The Principles of Agile Methods

- **Customer involvement**

- Customers should be closely involved throughout the development process.
- Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.

- **Incremental delivery**

- The software is developed in increments with the customer specifying the requirements to be included in each increment.

The Principles of Agile Methods

- **People not process**

- The skills of the development team should be recognized and exploited.
- Team members should be left to develop their own ways of working without prescriptive processes.

- **Embrace change**

- Expect the system requirements to change and so design the system to accommodate these changes.

- **Maintain simplicity**

- Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

The Principles of Agile Methods

- **Product development**

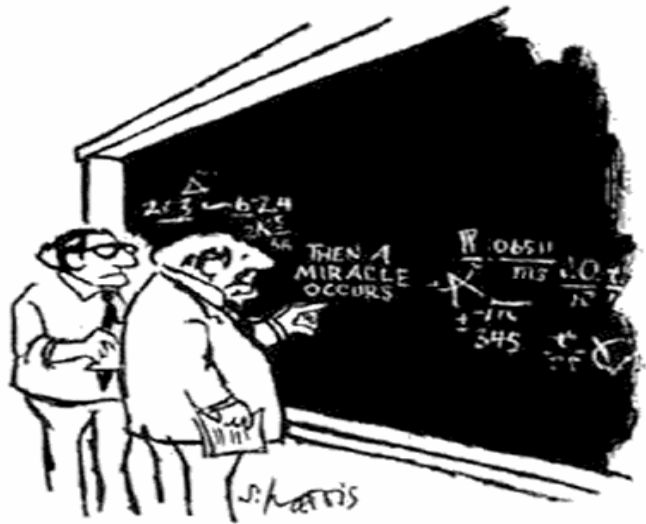
- A software company is developing a small or medium-sized product for sale.
 - Virtually all software products and apps are now developed using an agile approach

- **Custom system development within an organization**

- Commitment from the customer to become involved in the development process.
- Few external rules and regulations that affect the software.

Agile Method Applicability

- And now...?
- The Agile Manifesto does not tell us about Methods and Processes



Known agile

Methods:

- › Pair Programming
- › Refactoring
- › Personas
- › Continuous Integration
- › ...

Known agile

Frameworks:

- › XP
- › FDD
- › Crystal
- › Scrum
- › ...

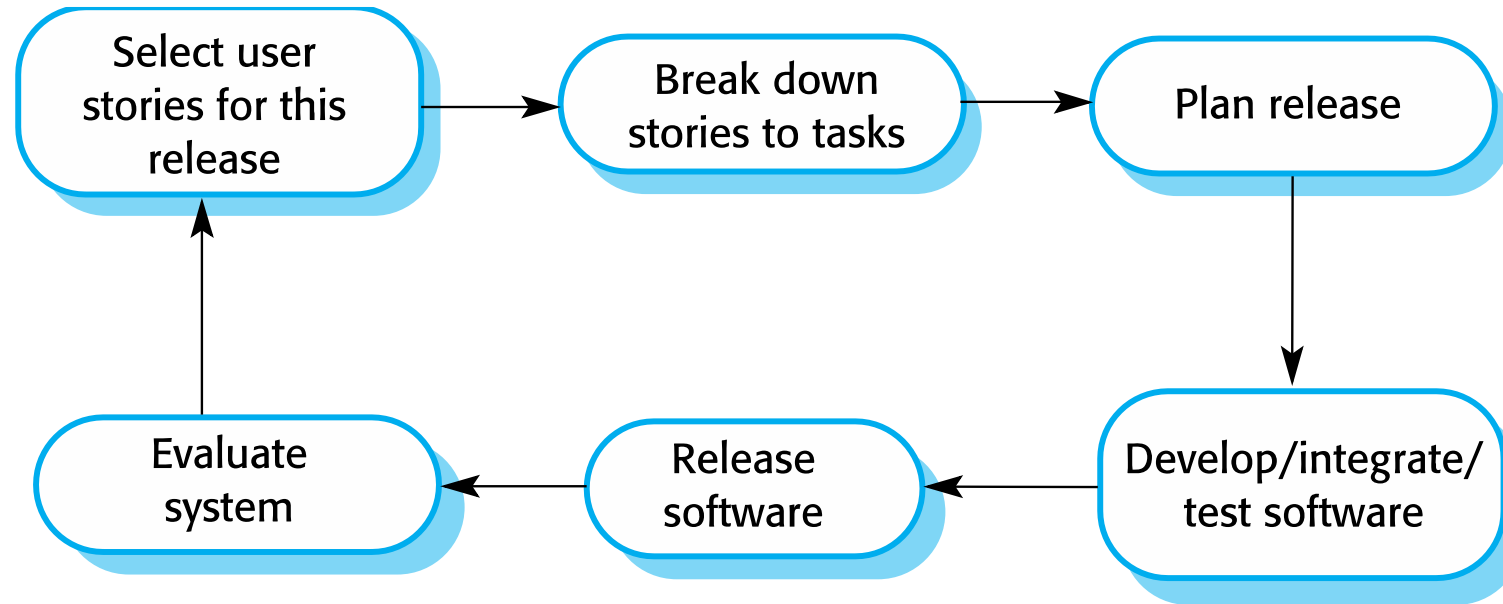
Agile Development Frameworks



Extreme Programming

- Developed in the late 1990s, that introduced a range of agile development techniques.
- Extreme Programming (XP) takes an 'extreme' approach to iterative development.
 - New versions may be built several times per day;
 - Increments are delivered to customers every 2 weeks;
 - All tests must be run for every build and
 - the build is only accepted if tests run successfully.

The Extreme Programming Release Cycle



Extreme Programming Practices

Extreme Programming - Practices



Stories

- "Requirements", but not mandatory
- Name + short story
- On index cards



Continuous Integration

- Integrate and test often
- Automated build system
- Automated regression tests



Refactoring

- Behavior preserving transformation
- Tool support, e.g. Eclipse



Pair Programming

- Focus on task
- Clarify ideas
- Rotate frequently



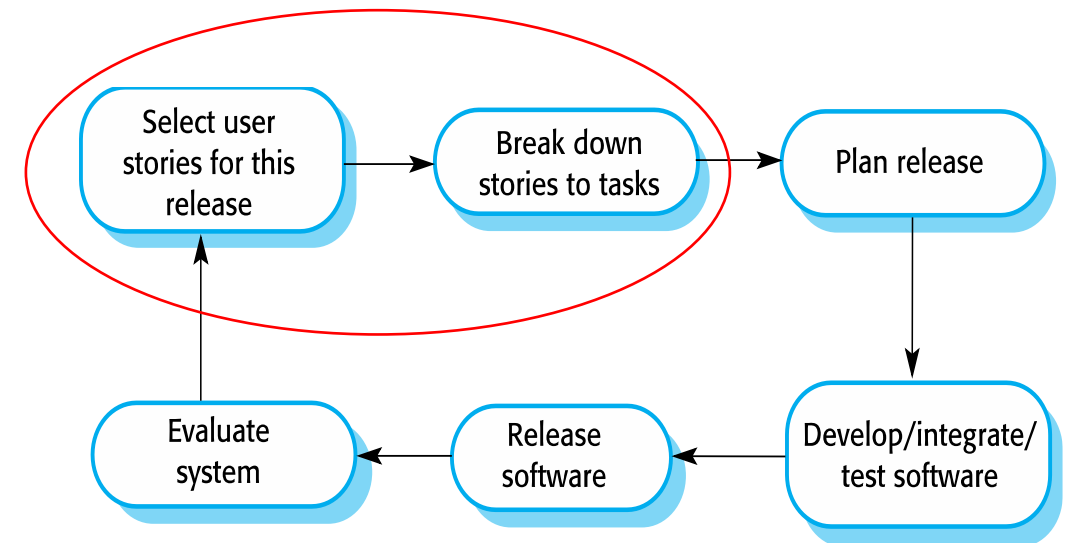
Test-First Programming

- Create tests before code
- Focus on interface and "what is needed"
- Gets tests for free

Extreme Programming Practices

- **Incremental planning**

- Requirements are recorded on story cards
- The stories to be included in a release are determined by :
 - the time available
 - their relative priority.
- The developers
 - break these stories into development 'Tasks'.
 - See Figures 3.5 and 3.6 in the book.



Extreme Programming Practices (cont.)

- **Small releases**

- The minimal useful set of functionality that provides business value is developed first.
- Releases of the system are frequent and incrementally add functionality to the first release.

- **Simple design**

- Enough design is carried out to meet the current requirements and no more.

- **Test-first development**

- An automated unit test framework is used to write tests for a new piece of functionality
- before that functionality itself is implemented.

Extreme Programming Practices (cont.)

- **Refactoring**

- All developers are expected to refactor the code continuously as soon as possible code improvements are found.
- This keeps the code simple and maintainable.

- **Pair programming**

- Developers work in pairs, checking each other's work and providing the support to always do a good job.

- **Collective ownership**

- The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers take responsibility for all of the code. Anyone can change anything.

Extreme Programming Practices (cont.)

- **Continuous integration**

- As soon as the work on a task is complete, it is integrated into the whole system.
- After any such integration, all the unit tests in the system must pass.

- **Sustainable pace**

- Large amounts of overtime are not considered acceptable as the net effect is often to reduce code quality and medium term productivity

- **On-site customer**

- A representative of the end-user of the system (the customer) should be available full time for the use of the XP team.
- In an extreme programming process, the customer is a member of the development team
 - responsible for bringing system requirements to the team for implementation.

XP and Agile Principles

- **Incremental development** is supported through
 - small, frequent system releases.
- **Customer involvement means**
 - full-time customer engagement with the team.
- **People not process** through
 - pair programming,
 - collective ownership
 - a process that avoids long working hours.
- **Change supported** through regular system releases.
- **Maintaining simplicity** through constant refactoring of code.

Influential XP Practices

- Extreme programming has a technical focus
 - is not easy to integrate with management practice in most organizations.
- Consequently,
 - while agile development uses practices from XP,
 - the method as originally defined is **not widely used**.
- Key practices
 - **User stories for specification**
 - **Refactoring**
 - **Test-first development**
 - **Pair programming**

User Stories for Requirements

- In XP, a customer or user is
 - part of the XP team
 - She/he is responsible for making decisions on requirements.
- User requirements are expressed as user stories or scenarios.
- These are
 - written on cards
 - the development team break them down into implementation tasks.
 - These tasks are the basis of schedule and cost estimates.
- The customer chooses the stories for inclusion in the next release based on
 - their priorities
 - the schedule estimates.

A 'Prescribing Medication' Story

Prescribing medication

The record of the patient must be open for input. Click on the medication field and select either 'current medication', 'new medication' or 'formulary'.

If you select 'current medication', you will be asked to check the dose; If you wish to change the dose, enter the new dose then confirm the prescription.

If you choose, 'new medication', the system assumes that you know which medication you wish to prescribe. Type the first few letters of the drug name. You will then see a list of possible drugs starting with these letters. Choose the required medication. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

If you choose 'formulary', you will be presented with a search box for the approved formulary. Search for the drug required then select it. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

In all cases, the system will check that the dose is within the approved range and will ask you to change it if it is outside the range of recommended doses.

After you have confirmed the prescription, it will be displayed for checking. Either click 'OK' or 'Change'. If you click 'OK', your prescription will be recorded on the audit database. If you click 'Change', you reenter the 'Prescribing medication' process.

Task Cards For Prescribing Medication

Task 1: Change dose of prescribed drug

Task 2: Formulary selection

Task 3: Dose checking

Dose checking is a safety precaution to check that the doctor has not prescribed a dangerously small or large dose.

Using the formulary id for the generic drug name, lookup the formulary and retrieve the recommended maximum and minimum dose.

Check the prescribed dose against the minimum and maximum. If outside the range, issue an error message saying that the dose is too high or too low. If within the range, enable the 'Confirm' button.

Refactoring (Why?)

- As time passes, our code becomes harder and harder to understand and maintain.
- The developers who touch the code come-and-go,
- Functionality evolves
 - which causes much more flows
- Technologies change but each of them leaves its signature on the code-base.
- Refactoring is our way, as engineers, to clean up the mess.

Refactoring

- Design for change: Conventional wisdom in software engineering.
- It is worth spending time and effort anticipating changes
 - as this reduces costs later in the life cycle.
- XP, however, maintains that
 - this is not worthwhile
 - as changes cannot be reliably anticipated
- Rather,
 - it proposes constant code improvement (refactoring)
 - to make changes easier when they have to be implemented.

Refactoring - Benefits

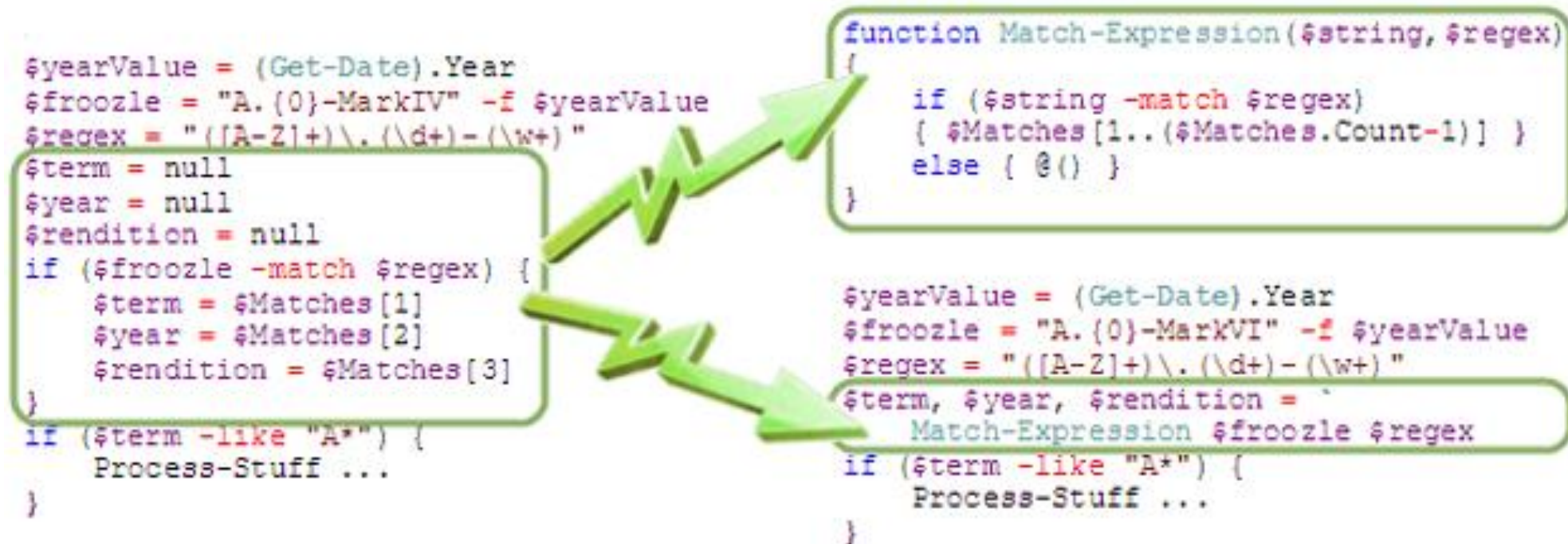
- Programming team look for possible software improvements
 - make these improvements even where there is no immediate need for them.
 - This improves the understandability of the software
 - so reduces the need for documentation.
- Changes are easier to make
 - because the code is well-structured and clear.
- However, some changes requires architecture refactoring
 - this is much more expensive.

Examples of Refactoring

- Re-organization of a class hierarchy to remove duplicate code.
- Tidying up and renaming attributes and methods to make them easier to understand.
- The replacement of inline code with calls to methods that have been included in a program library.

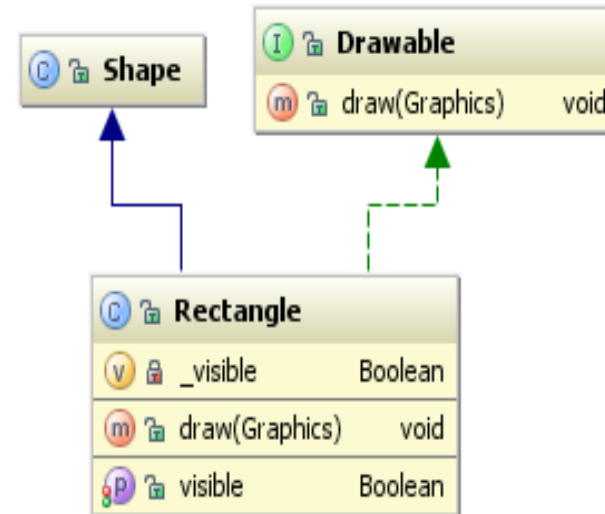
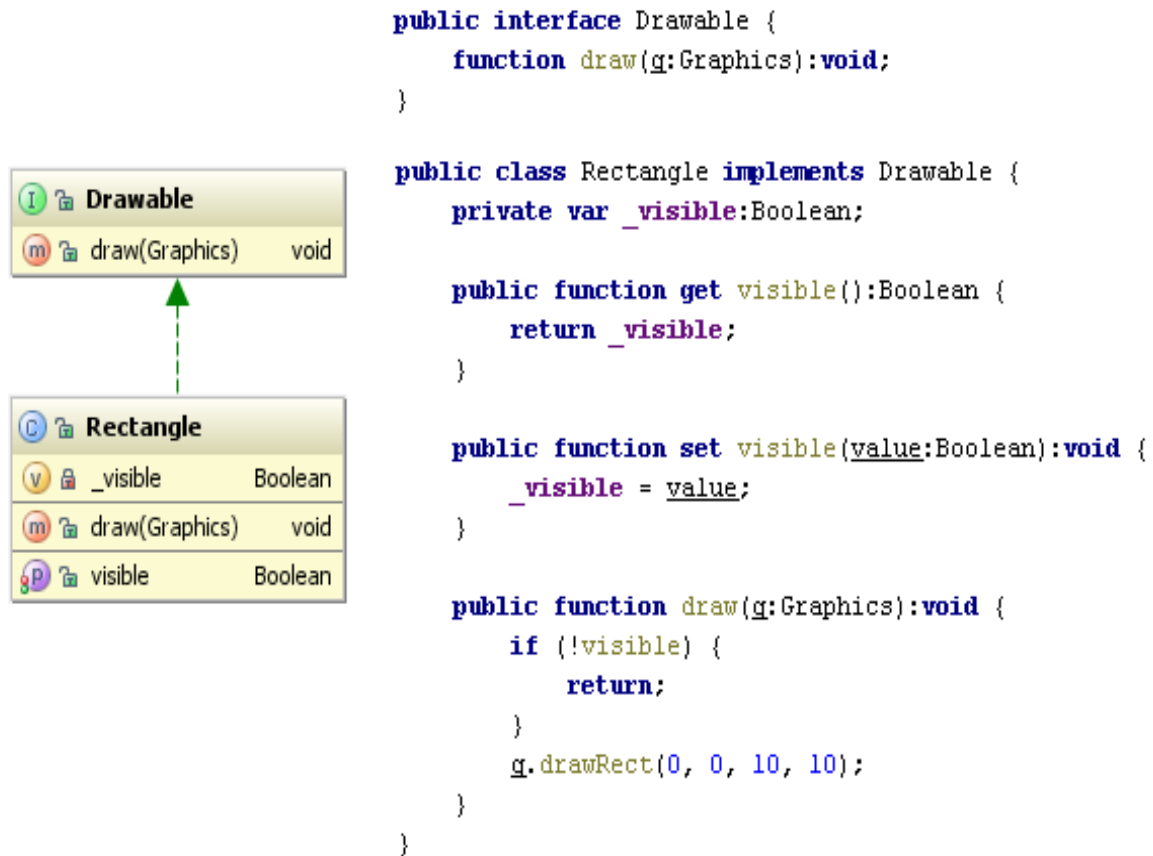
Examples of Refactoring

- The replacement of inline code with calls to methods that have been included in a program library.



Examples of Refactoring

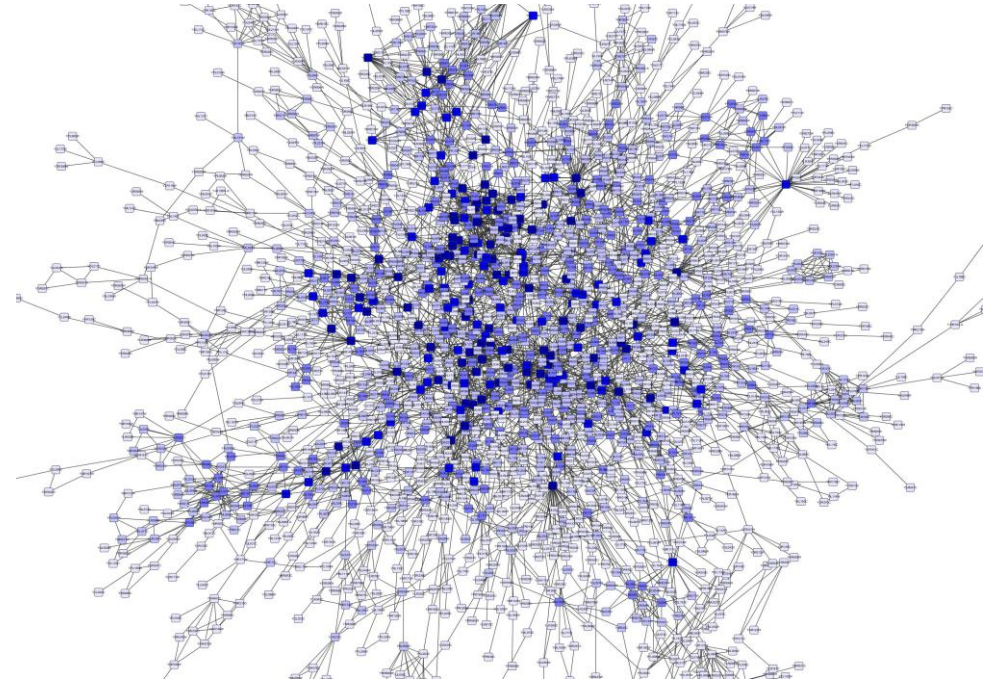
- Re-organization of a class hierarchy to remove duplicate code.



```
public interface Drawable {  
    function draw(g:Graphics):void;  
}  
  
public class Shape {  
}  
  
public class Rectangle  
    extends Shape implements Drawable {  
  
    private var _visible:Boolean;  
  
    // ....  
}
```

Test-first Development

- Technique for designing code
 - Test first not the production code
 - Focusing on the outside behavior or the interface
 - Write the test and it fails.
 - That is good because we have not written the production code
 - Write enough code to make the test pass
 - Code coverage



Test-driven Development (in XP)

- Writing tests before code
 - clarifies the requirements to be implemented.
- Tests:
 - are written as programs rather than data so that they can be executed automatically.
 - include a check that it has executed correctly.
 - rely on a testing framework such as Junit.
- All previous and new tests are run automatically when new functionality is added,
 - thus checking that the new functionality has not introduced errors.

Test-driven Development (in XP)

- The role of the customer in the testing process is to
 - help develop acceptance tests for the stories
- The customer who is part of the team
 - Writes tests as development proceeds.
- However:
 - people adopting the customer role have limited time available
 - so cannot work full-time with the development team.
 - They may feel that providing the requirements was enough of a contribution and so may be reluctant to get involved in the testing process.

Test Case Description For Dose Checking

Test 4: Dose checking

Input:

1. A number in mg representing a single dose of the drug.
2. A number representing the number of single doses per day.

Tests:

1. Test for inputs where the single dose is correct but the frequency is too high.
2. Test for inputs where the single dose is too high and too low.
3. Test for inputs where the single dose * frequency is too high and too low.
4. Test for inputs where single dose * frequency is in the permitted range.

Output:

OK or error message indicating that the dose is outside the safe range.

Test automation

- Test automation means that tests are written as executable components before the task is implemented
 - These testing components
 - should be stand-alone
 - should simulate the submission of input to be tested
 - should check that the result meets the output specification.
 - An automated test framework (e.g. Junit) is a system that makes it easy to
 - write executable tests
 - submit a set of tests for execution.

Test automation

- Whenever any functionality is added to the system,
 - the tests can be run
 - problems that the new code has introduced can be caught immediately.

Problems with Test-first Development

- Programmers prefer programming to testing
 - sometimes they take short cuts when writing tests.
 - For example,
 - they may write incomplete tests that do not check for all possible exceptions that may occur.
- Some tests can be very difficult to write incrementally.
 - For example,
 - in a complex user interface, it is often difficult to write unit tests for the code that
 - implements the 'display logic'
 - workflow between screens.
- It is difficult to judge the completeness of a set of tests.
- Although you may have a lot of system tests,
 - your test set may not provide complete coverage.

Pair Programming

- Pair programming involves
 - programmers working in pairs,
 - developing code together.
- This helps develop
 - common ownership of code
 - spreads knowledge across the team.
- It serves as
 - an informal review process as each line of code is looked at by more than 1 person.
- As the whole team can benefit from improving the system code.



a pair working together is more efficient than 2 programmers working separately, **Why?**

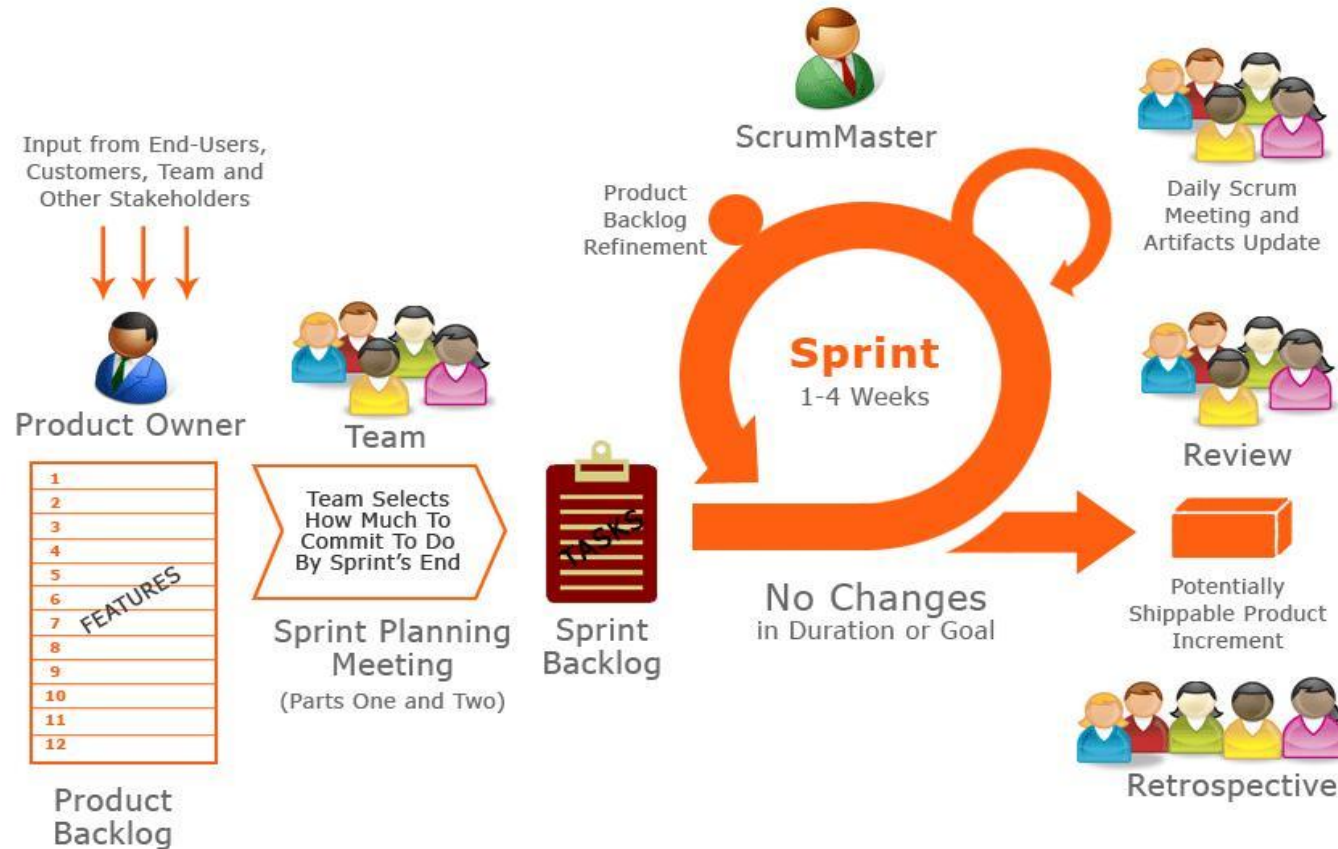
Agile Project Management



Scrum

- Scrum is an agile method (implementation) that focuses on
 - managing iterative development
 - rather than specific agile practices (XP).

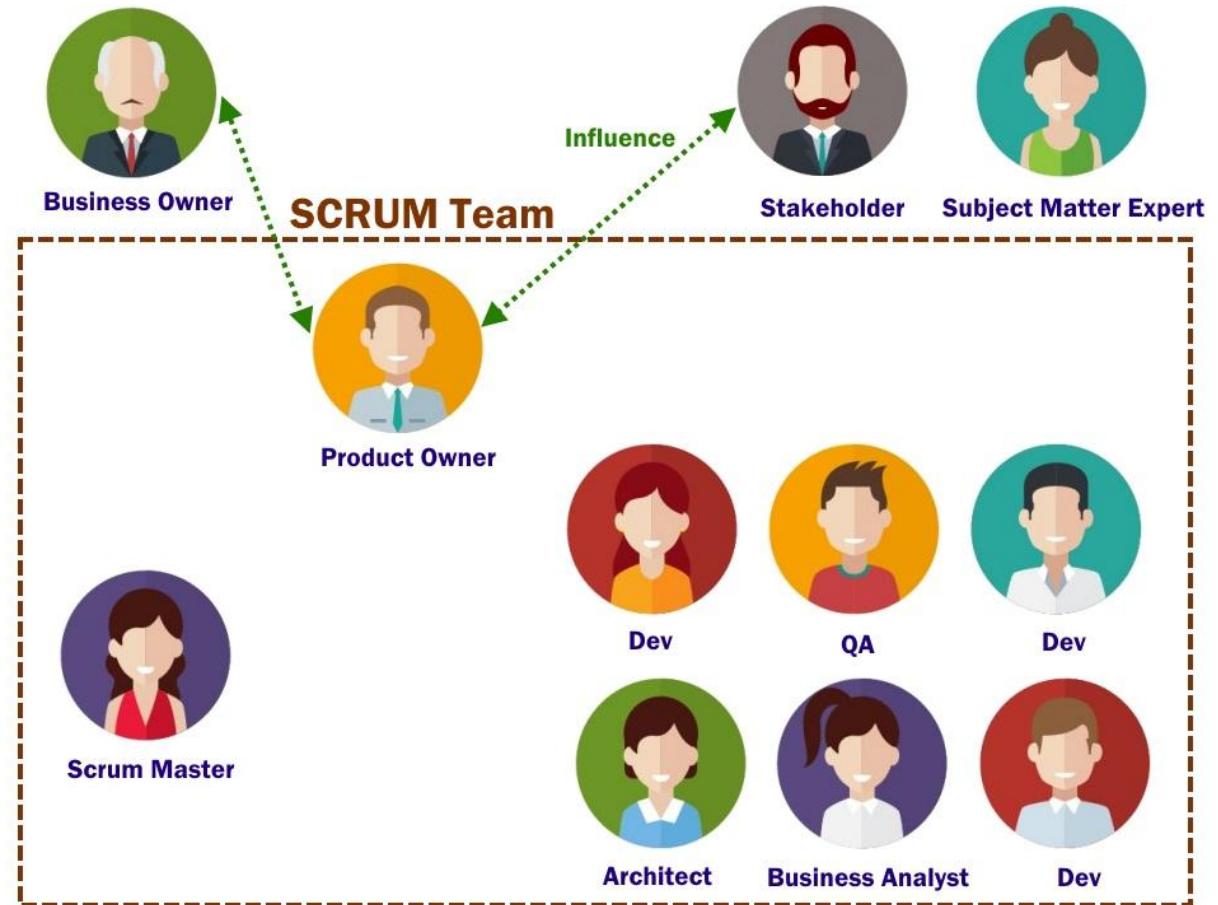
Overview of Scrum



Scrum terminology

- **Scrum Team**

- Product owner
- ScrumMaster
- Development team

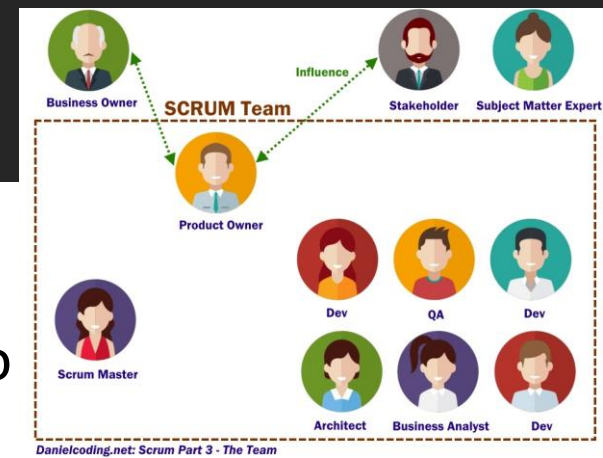


Danielcoding.net: Scrum Part 3 - The Team

Scrum terminology (cont.)

- **Product owner**

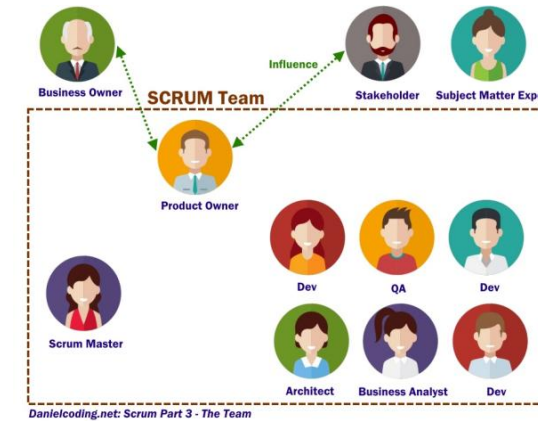
- An individual (or possibly a small group) whose job is to
 - identify product features or requirements,
 - prioritize these for development
 - continuously review the product backlog
 - to ensure that the project continues to meet critical business needs.
- The Product Owner can be
 - a customer
 - a product manager in a software company
 - other stakeholder representative



Scrum terminology (cont.)

- **ScrumMaster**

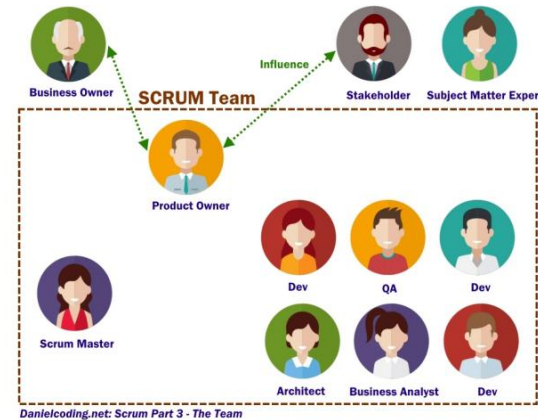
- The ScrumMaster is
 - responsible for ensuring the Scrum process is followed
 - guides the team in the effective use of Scrum
 - responsible for interfacing with the rest of the company
 - ensuring that the Scrum team is not diverted by outside interference.



Scrum terminology (cont.)

- **Development team**

- A self-organizing group of software developers,
 - which should be no more than 7 people.
- They are responsible for
 - developing the software
 - other essential project documents.



Scrum terminology (cont.)

- **Scrum**

- A daily meeting of the Scrum team
 - that reviews progress and prioritizes work to be done that day.
- Ideally, this should be a short face-to-face meeting that includes the whole team.

- **Sprint**

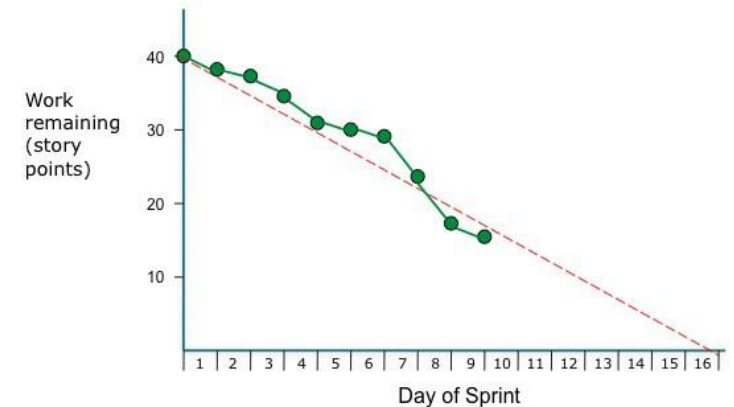
- A development iteration.
- Sprints are usually 2-4 weeks long.

Scrum terminology (cont.)

- **Velocity**

- An estimate of how much product backlog effort that a team can cover in a single sprint.
- Understanding a team's velocity helps them estimate
 - what can be covered in a sprint
 - provides a basis for measuring improving performance.

Velocity is Plotted on the "Burndown Chart"



Burndown chart answers the question, "Are we on track to successfully deliver this sprint's output?"

scruminc.

© 2012 Scrum Inc.

Scrum terminology (cont.)

- **Product backlog**

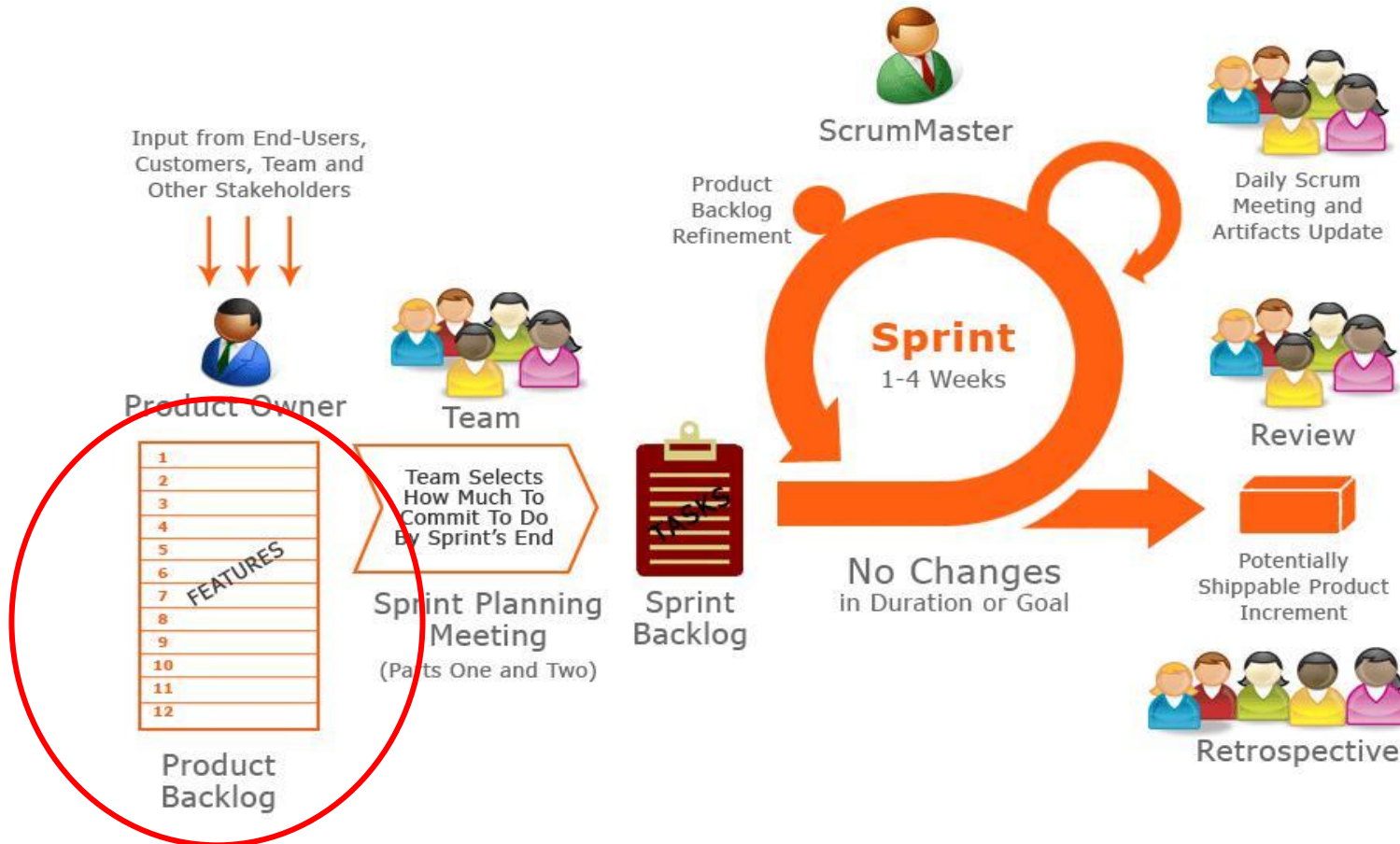
- This is a list of 'to do' items which the Scrum team must tackle.
- They may be
 - feature definitions for the software,
 - software requirements,
 - user stories or descriptions of supplementary tasks
 - user documentation.

- **Potentially shippable product increment**

- The software increment that is delivered from a sprint.

Scrum Sprint Cycle

- The starting point for planning is the product backlog,
 - which is the list of work to be done on the project



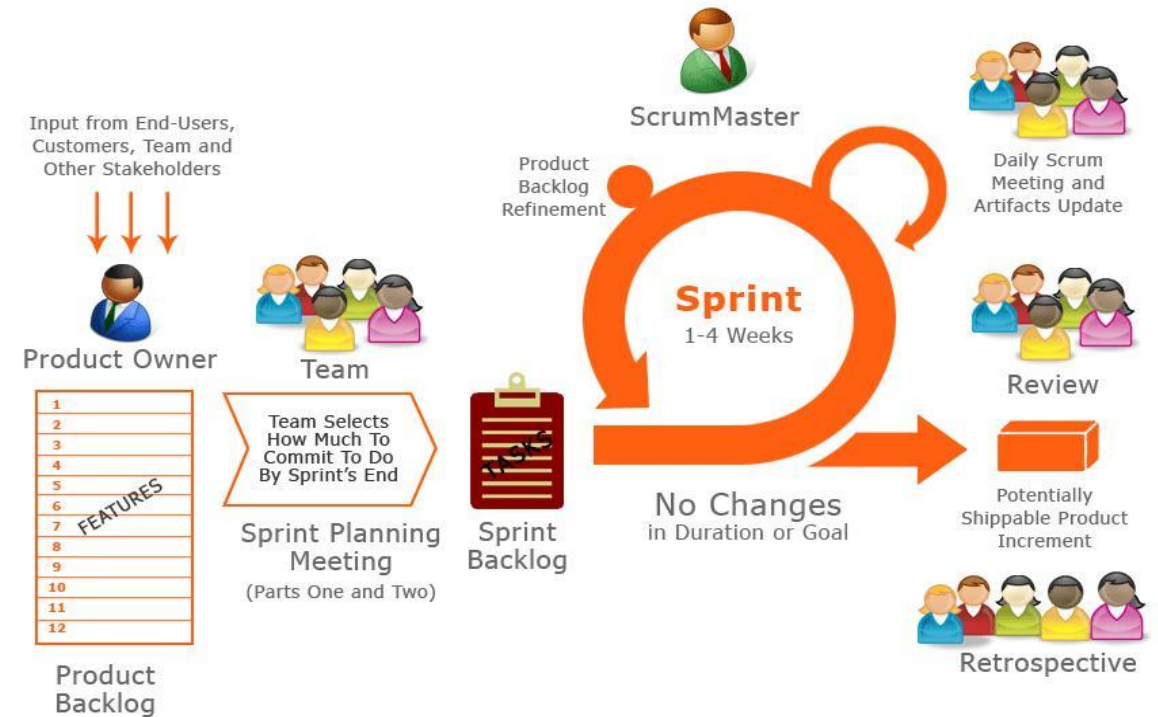
Scrum Sprint Cycle

Product Backlog Example

Order	Work Item Type	PBI	Title
1	Product Backlog Item		Patient Data should be saved
	Task		Create GUI for patient Data
	Task		Create Class for Saving Data
	Task		Create Database Table
2	Product Backlog Item		The Nurse needs Medical Information
	Task		Create GUI that list Medical Prescriptions for a specific Patient
3	Product Backlog Item		The Doctor should get Reports
	Task		Create Search GUI
	Task		Create search Class and Methods
	Task		Create Report "Get Patient Data"

Scrum Sprint Cycle

- Sprints are fixed length, normally 2–4 weeks.
 - The selection phase involves all of the project team (working with customer) to
 - select the features and functionality from tl



Scrum Sprint Cycle

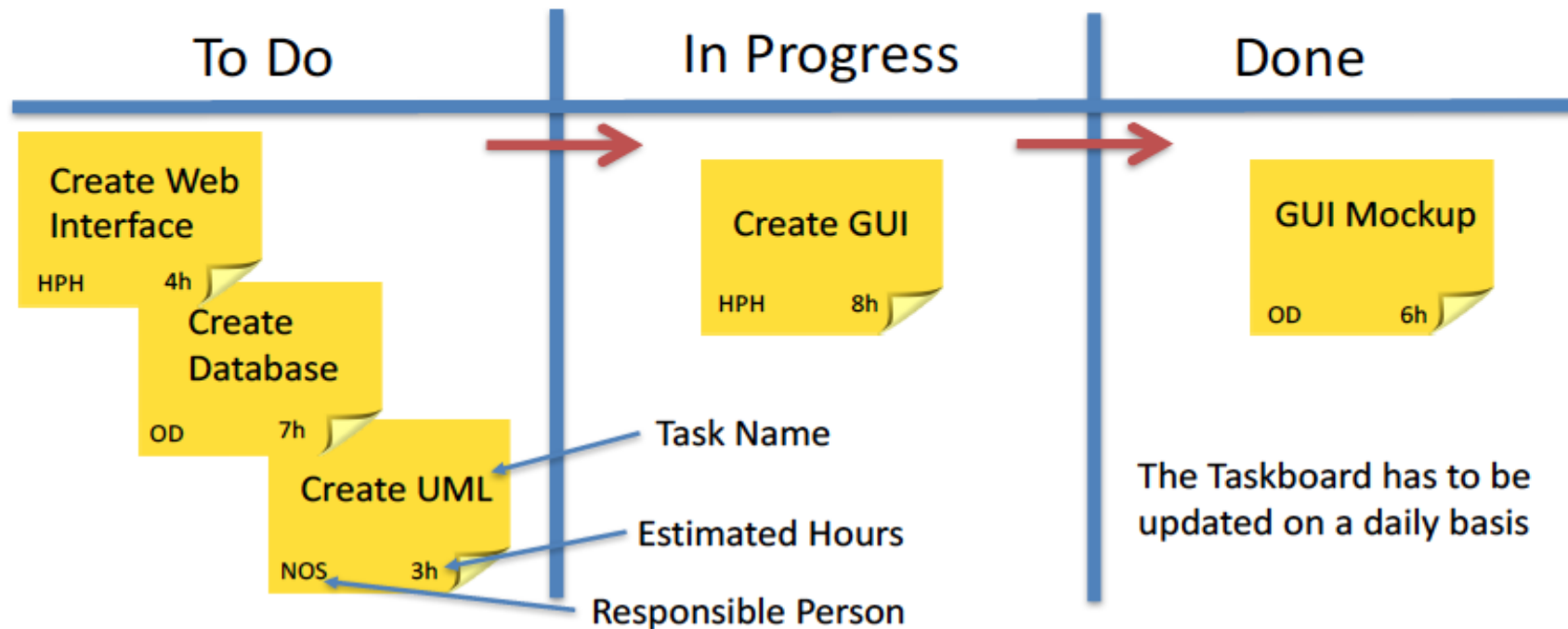
The Sprint Backlog

The screenshot shows the Visual Studio Online interface for a 'Software Development Team Sprint 1'. The top navigation bar includes 'HOME', 'CODE', 'WORK', 'BUILD', and 'TEST'. Below this, the 'Backlogs' tab is selected, showing a list of sprints under 'Current' and 'Future' categories. 'Sprint 1' is the active sprint. The main area displays the 'Backlog' view for this sprint, showing a list of tasks with columns for Title, State, Assigned To, and Remaining. Two tasks are listed: 'Create GUI for Web App' and 'Add Database Communication with Server', both in 'New' state.

Title	State	Assigned To	Rema...
Create GUI for Web App	New		
Add Database Communication with Server	New		

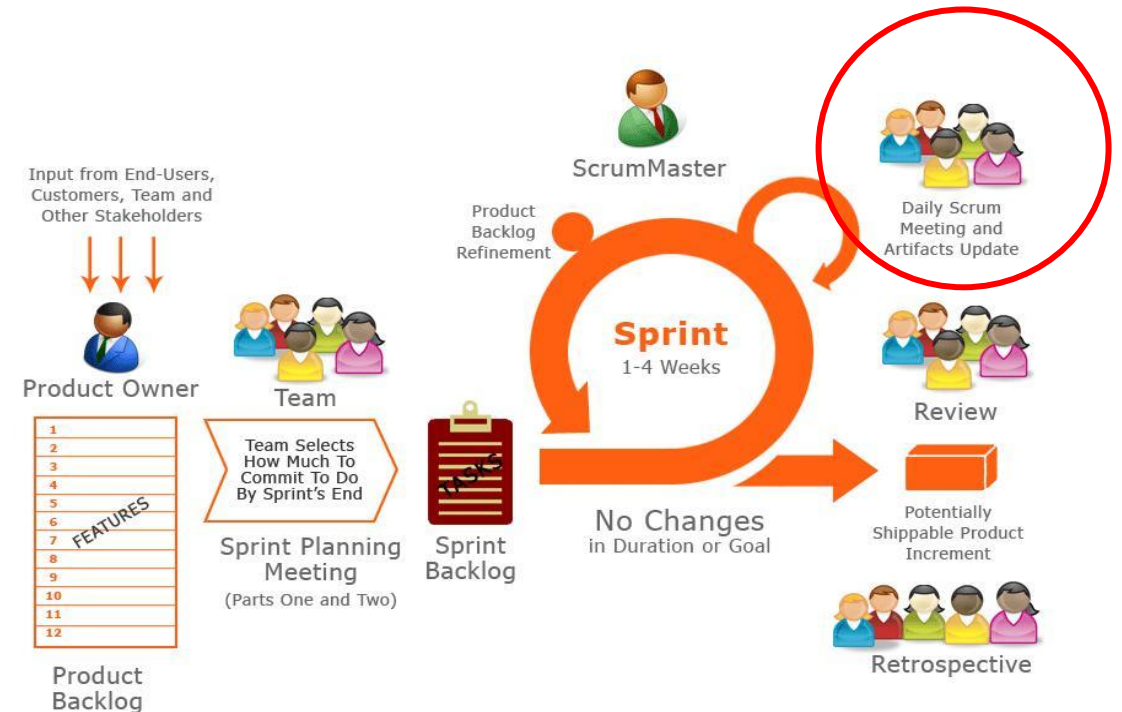
Scrum Sprint Cycle

Sprint Task Board Example



The Sprint Cycle

- Once features are agreed, the team organize themselves to develop the software.
- During this stage
 - The team is isolated from the customer and the organization,
 - All communications channelled through the so-called 'Scrum master'.
 - The role of the Scrum master is to protect the development team from external distractions.



The Sprint Cycle



Daily Scrum Meeting



3 Questions:

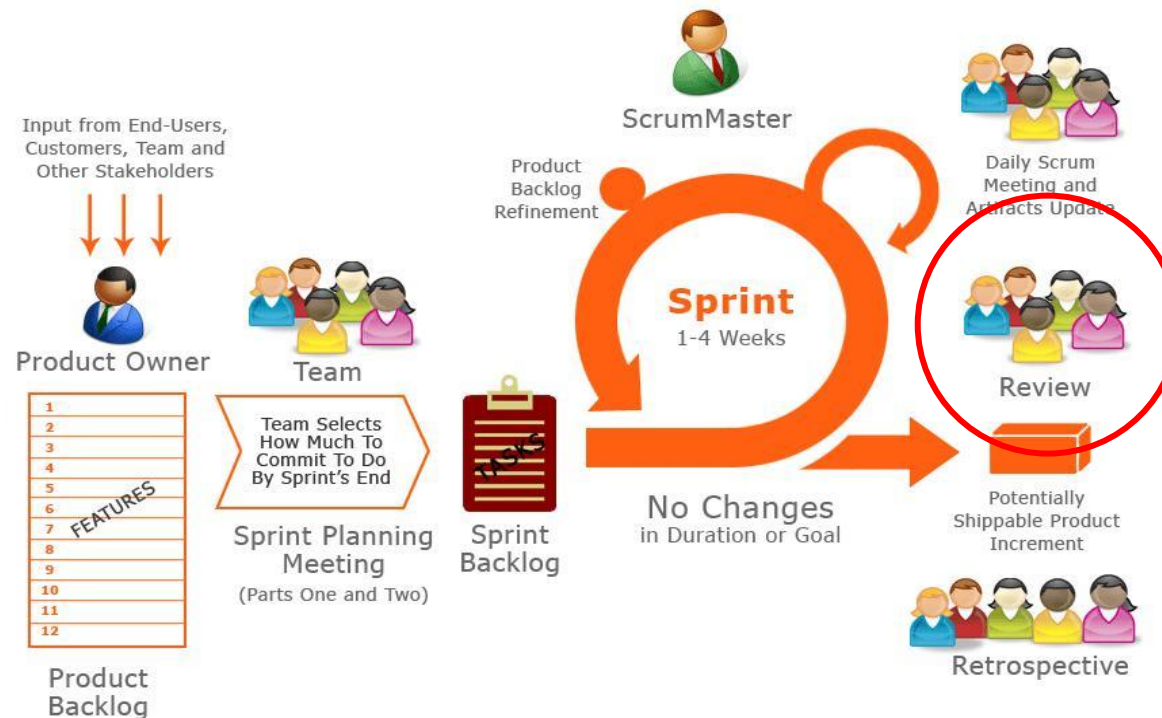
1. What did you do yesterday (since last meeting)?
2. What shall you do today (until the next meeting)?
3. Any Problems?

Purpose with the Daily Scrum Meeting:

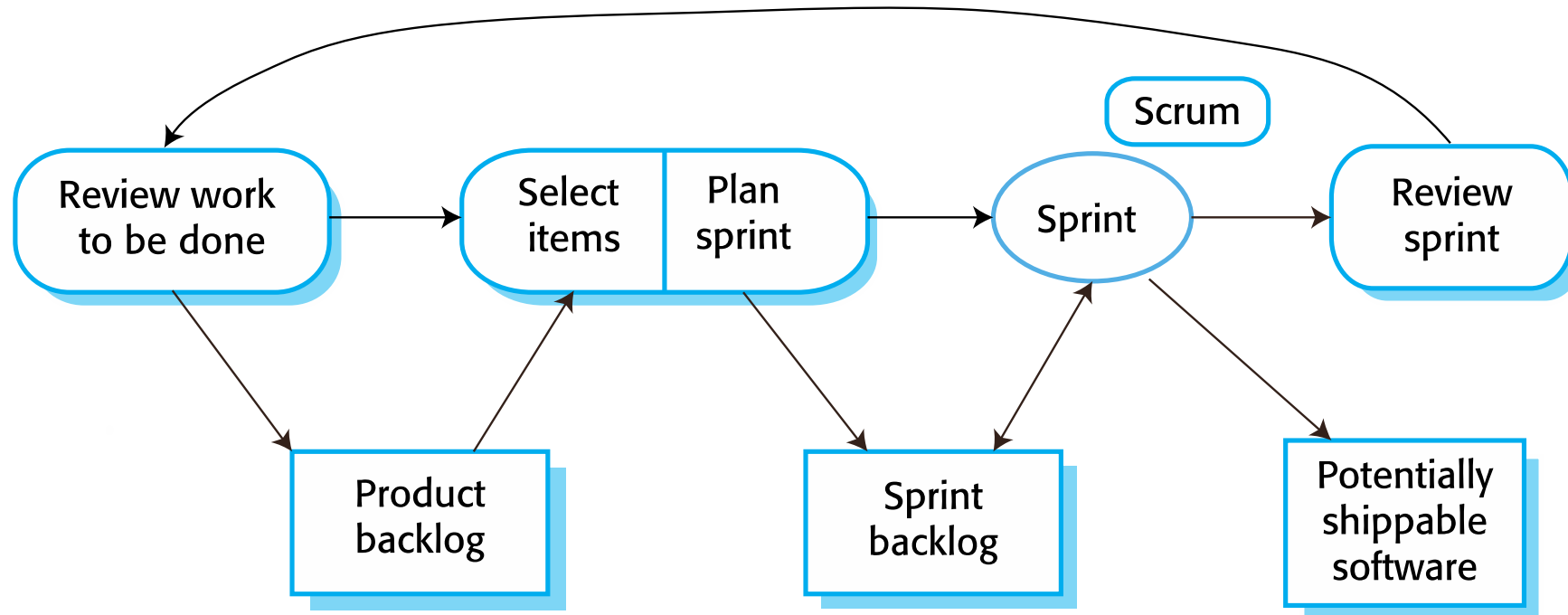
- Synchronize activities and create a plan for next 24 hours.
- Track Progress

The Sprint Cycle

- At the end of the sprint,
 - the work done is reviewed and presented to stakeholders.
 - The next sprint cycle then begins.



Scrum Sprint Cycle



Scrum Benefits

- The product is broken down into a set of manageable and understandable chunks.
- Unstable requirements do not hold up progress.
- The whole team
 - have visibility of everything and consequently team communication is improved.
- Customers see
 - on-time delivery of increments and gain feedback on how the product works.
- Trust between customers and developers is established and
 - a positive culture is created in which everyone expects the project to succeed.